

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

FACULTY: Dr. D.SUDHAGAR

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I **K Gowthami(192525428)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K Gowthami

QUESTIONS: 05

Total Marks:25

1. Error Analysis in Maximum Likelihood Estimation (MLE)

a) Identify the conceptual error.

The likelihood function was written as the sum of probabilities instead of the product of probabilities of all observations.

b) Effect on parameter estimation.

- Produces incorrect parameter estimates.
- The model does not maximize the true likelihood.
- Prediction accuracy decreases even with large datasets.

c) Correct mathematical formulation.

For independent observations:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

The product represents the joint probability of observing all data points.

d) How log-likelihood fixes computational issues.

- Converts products into sums.
- Prevents numerical underflow for very small probabilities.
- Makes optimization easier.

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

2. Error Diagnosis in Perceptron Learning

a) Possible implementation errors.

- Incorrect weight update rule.
- Wrong bias update.
- Incorrect learning rate.
- Wrong activation function.
- Errors in data labels.
- Not shuffling the training data.

b) Effect of incorrect learning rate or weight updates.

- Very small learning rate → Slow convergence.
- Very large learning rate → Overshoots the solution.
- Incorrect weight updates → Model fails to learn.

c) Dataset-related issues.

- Data is not linearly separable.
- Incorrect class labels.
- No feature scaling.
- Presence of noisy data.

d) Modifications to ensure convergence.

- Use a suitable learning rate.
- Normalize the input features.
- Verify correct labels.
- Shuffle training samples.
- Use enough training iterations.

3. Backpropagation Gradient Error Analysis

a) Possible mathematical or coding errors.

- Incorrect derivative calculation.
- Wrong gradient sign.
- Incorrect weight updates.
- Errors in loss function implementation.
- Mistakes in chain rule calculation.

b) Role of activation functions.

Activation functions introduce non-linearity and control gradient flow.

- Sigmoid may cause vanishing gradients.
- ReLU reduces vanishing gradient problems.

c) Vanishing and exploding gradients.

- **Vanishing gradients:** Gradients become very small, causing slow learning.
- **Exploding gradients:** Gradients become very large, causing unstable updates and increasing loss.

d) Techniques to fix the issue.

- Normalize input data.
- Use ReLU activation.
- Apply Xavier or He weight initialization.
- Use Batch Normalization.
- Apply Gradient Clipping.
- Reduce the learning rate.

4. Gradient Descent vs SGD Error Investigation

a) Why Batch Gradient Descent is slow.

- Uses the entire dataset for every update.
- Requires high computation time.
- Slow on large datasets.

b) Cause of fluctuations in SGD.

- Updates weights after every training sample.
- Produces noisy gradients.
- Loss fluctuates during training.

c) Error behavior of Mini-Batch Gradient Descent.

Method	Error Behavior
Batch Gradient Descent	Stable but slow
Stochastic Gradient Descent	Fast but noisy
Mini-Batch Gradient Descent	Faster and more stable

d) Recommended approach.

Mini-Batch Gradient Descent is recommended because:

- Faster than Batch Gradient Descent.
- More stable than SGD.
- Efficient for large datasets.
- Provides a good balance between speed and accuracy.

5. Curse of Dimensionality Error Analysis

a) How high dimensionality increases model error.

- Data becomes sparse.
- Model learns noise instead of useful patterns.
- Generalization performance decreases.
- Risk of overfitting increases.

b) Signs of overfitting.

- High training accuracy.
- Low testing accuracy.
- Large gap between training and testing performance.
- Poor prediction on unseen data.
-

c) Why distance-based learning fails in high dimensions.

- Distances between data points become very similar.
- Nearest neighbors become difficult to distinguish.
- Similarity measures lose effectiveness.
- Algorithms such as **KNN** and **K-Means** perform poorly in high-dimensional spaces.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand